

LAB : L'état et le contexte d'un agent

Table des matières

PARTIE 1 : Définir une classe de contexte structurée appelée ColourContext.....	2
PARTIE 2 : Agent sans contexte.....	
2 PARTIE 3 :	Agent avec
contexte.....	2 PARTIE 4 :
Changement de contexte.....	3 PARTIE 5
: Définir un état personnalisé d'agent en héritant de AgentState.....	3 PARTIE 6
: Agent qui modifie un état	4 PARTIE
6 : Agent qui récupère un état.....	5

SMA et IAD Master SDIA Prof. RETAL SARA

PARTIE 1 : Définir une classe de contexte structurée appelée ColourContext

```
from dataclasses import dataclass

@dataclass
class ColourContext:
    favourite_colour: str = "blue"
    least_favourite_colour: str = "yellow"
```

PARTIE 2 : Agent sans contexte

```
from langchain_ollama import ChatOllama
from langchain.agents import create_agent
model = ChatOllama(
    model="llama3.2:3b", # ou mistral, gemma, etc.
```

```
)
agent = create_agent(model=model,
context_schema=ColourContext)
```

```
from langchain.messages import HumanMessage
```

```
response = agent.invoke(
    {"messages": [HumanMessage(content="What is my favourite
    colour?")]}], context=ColourContext())
print(response['messages'][-1].content)
```

PARTIE 3 : Agent avec contexte

```
from langchain.tools import tool, ToolRuntime
```

```
@tool
def get_favourite_colour(runtime: ToolRuntime) -> str:
    """Get the favourite colour of the user"""
    return runtime.context.favourite_colour
SMA et IAD Master SDIA Prof. RETAL SARA
```

```
@tool
def get_least_favourite_colour(runtime: ToolRuntime) ->
str: """Get the least favourite colour of the user"""
    return runtime.context.least_favourite_colour
```

```
agent = create_agent(
    model=model,
    tools=[get_favourite_colour,
    get_least_favourite_colour],
    context_schema=ColourContext
)
response = agent.invoke(
    {"messages": [HumanMessage(content="What is my favourite
    colour?")]}], context=ColourContext())
print(response['messages'][-1].content)
```

PARTIE 4 : Changement de contexte

```

response = agent.invoke(
    {"messages": [HumanMessage(content="What is my favourite
    colour?")]}], context=ColourContext(favourite_colour="green")
)
print(response['messages'][-1].content)

```

PARTIE 5 : Définir un état personnalisé d'agent en héritant de AgentState

AgentState est une structure de base utilisée par les agents LangChain

CustomState est notre état enrichi

```

from langchain.agents import AgentState

```

```

class CustomState(AgentState):
    favourite_colour: str

```

SMA et IAD Master SDIA Prof. RETAL SARA

PARTIE 6 : Agent qui modifie un état

```

from langchain.tools import tool, ToolRuntime
from langgraph.types import Command
from langchain.messages import ToolMessage

```

```

@tool
def update_favourite_colour(favourite_colour: str, runtime: ToolRuntime) ->
Command:
    """Update the favourite colour of the user in the state once they've re
vealed it."""
    return Command(update={
        "favourite_colour": favourite_colour,
        "messages": [ToolMessage("Successfully updated favourite colour",
tool_call_id=runtime.tool_call_id)]})

```

```

from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver

```

```

agent = create_agent(

```

```

        model=model,
        tools=[update_favourite_colour],
        checkpointer=InMemorySaver(),
        state_schema=CustomState
    )

from langchain.messages import HumanMessage

response = agent.invoke(
    { "messages": [HumanMessage(content="My favourite colour is
    green")]}], {"configurable": {"thread_id": "1"}}
)
print(response['messages'][-1].content)

```

SMA et IAD Master SDIA Prof. RETAL SARA

PARTIE 6 : Agent qui récupère un état

```

@tool
def read_favourite_colour(runtime: ToolRuntime) -> str:
    """Read the favourite colour of the user from the
    state.""" try:
        return runtime.state["favourite_colour"]
    except KeyError:
        return "No favourite colour found in state"

agent = create_agent(
    model=model,
    tools=[update_favourite_colour, read_favourite_colour],
    checkpointer=InMemorySaver(),
    state_schema=CustomState
)

response = agent.invoke(
    { "messages": [HumanMessage(content="My favourite colour is
    green")]}], {"configurable": {"thread_id": "1"}}
)
print(response['messages'][-1].content)

response = agent.invoke(
    { "messages": [HumanMessage(content="What's my favourite
    colour?")]}], {"configurable": {"thread_id": "1"}}
)
print(response['messages'][-1].content)

```